



ENME441

Brokered Communication

MQTT + HiveMQ user interfacing

Topics



PART 1

Brokered comms

- Why peer-to-peer fails
- MQTT publish / subscribe model
- hivemq.py on the ESP32



PART 2

User interfacing

- Interface options for brokered comms
- MQTT over WebSocket
- Javascript template

01

Brokered comms

Communicating through a cloud MQTT broker



umd-iot Isolates Everything

Your ESP32 can access data from other internet servers, but nothing can reach you from the outside – all inbound connections are blocked.

1

Cross-subnet isolation

Devices on umd-iot cannot talk to devices on eduroam, and vice versa. Your laptop cannot reach your ESP32 directly.

2

Peer-to-peer isolation

Two ESP32s on umd-iot cannot reach each other either, even sitting side by side on the same network.

3

All inbound connections blocked

Even knowing the IP, you cannot SSH in, send it an HTTP request, or open a socket to it from outside.

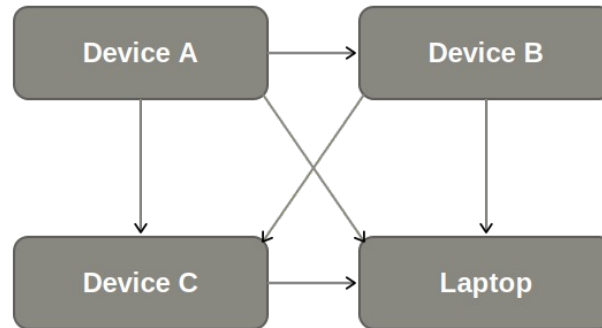
4

Outbound connections are allowed

This one exception is the whole basis of the architecture we're about to build.

Peer-to-Peer vs. Brokered Communications

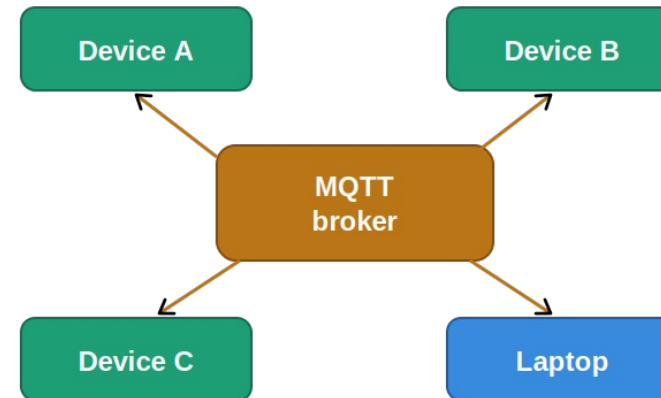
Peer-to-peer (problematic)



Issues:

- Every device must know every other
- $N \text{ devices} = N \times (N-1)$ connections
- Requires open inbound ports
- Blocked by NAT and firewalls
- No message persistence

Broker-based pub/sub (scalable)



Benefits:

- Devices only connect to one endpoint
- $N \text{ devices} = N$ connections
- Only outbound connections needed
- Works through NAT and firewalls
- Broker stores messages if device offline

How publish/subscribe works

Publish: A device sends a message to a named topic (e.g., "course/student42/temperature")

Subscribe: Any client registers interest in a topic; the broker delivers matching messages

Decouple: Publishers and subscribers don't know about each other — only the broker does

Persist: The broker can hold retained messages so new subscribers get the last known value

MQTT: The Pub/Sub Protocol for IoT

Message Query Telemetry Transport (MQTT) allows a broker to manage bidirectional communication between publishers and subscribers.

1

Publish / subscribe to selected “topics”

ESP32 publishes data to a topic, and a laptop subscribes to the same topic to get messages. Because the ESP32 initiates the communication, messages can be returned to the ESP32 by the broker, even on networks that block inbound connections.

2

Built for constrained devices

Message headers as small as 2 bytes, keep-alive and reconnect built in, client library with small memory footprint.

3

Quality of service and retained messages

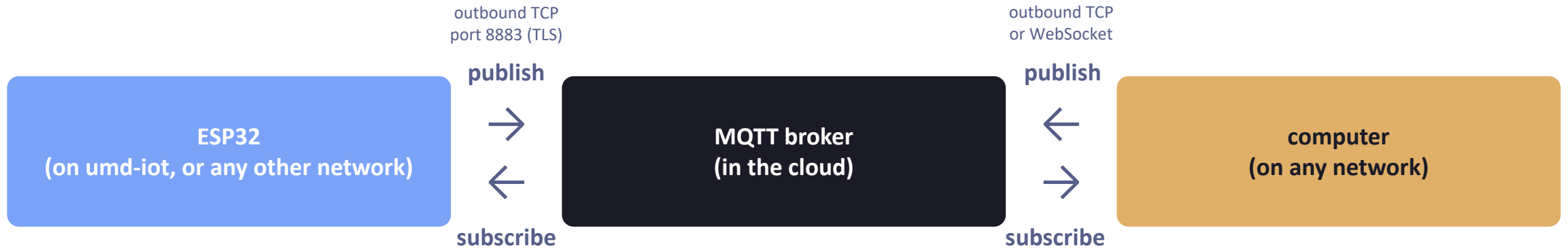
QoS levels guarantee delivery; a retained message lets a device that just reconnected immediately learn the current state.

4

Last Will and Testament

You register an 'offline' message at connect time and the broker publishes it for you if your device drops unexpectedly.

One-Way Connection, Bi-Directional Messaging



- **Both sides connect with the MQTT broker**

Neither device ever accepts an incoming connection, so no firewall or NAT rules are broken.

- **Keep-alive holds it open**

The MQTT protocol pings periodically so the connection survives indefinitely.

- **TCP is bidirectional**

Once an MQTT connection is open, the broker can push messages to connected nodes at any time.

HiveMQ Cloud — ENME441 MQTT Broker

HiveMQ is a cloud-based MQTT broker.

1

A broker is a program, not a protocol

MQTT defines the message format. The broker is a program that speaks the MQTT protocol, sits at a fixed address, holds every client's connection open, and routes messages in both directions.

2

The broker must live where both sides can reach it

You can run your own MQTT server (e.g. Mosquitto) on a laptop, but this puts the broker behind the same firewall and NAT that blocked you in the first place. A broker on the public internet is reachable from umd-iot, eduroam, or any other internet-connected network.

3

HiveMQ Cloud is a managed broker with a robust free tier

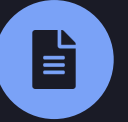
A public hostname, a valid TLS certificate, always-on uptime, and flexible MQTT messaging.

4

Any MQTT client can connect to HiveMQ

umqtt.simple on the ESP32, MQTT.js in a browser, MQTT Explorer on your desktop, paho-mqtt in a Python script. Same cluster, same credentials, same topics.

HiveMQ Setup



Follow the instructions in `brokered_comms_setup.pdf` to prepare your ESP32 and laptop for working with MQTT through the HiveMQ broker.

https://github.com/ddevoe-umd/enme441/blob/main/docs/brokered_comms_setup.pdf

Key steps:

- Install the MQTT library on your ESP32:

```
import mip
mip.install('umqtt.simple')
```

- Install the `hivemq.py` example code on your ESP32:

```
mip.install('github:ddevoe-umd/enme441/micropython/10_Brokered_Communication/hivemq.py')
```

- Install MQTT Explorer on your laptop:

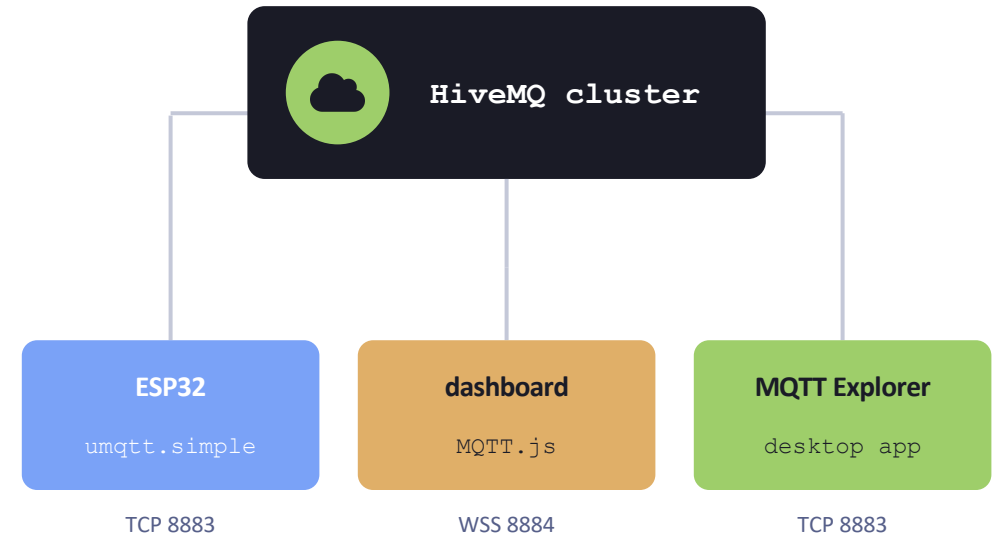
<https://mqtt-explorer.com>

One HiveMQ Cluster for the Entire Class

All class devices will connect to a single HiveMQ cluster with one shared set of credentials.

CONNECTION DETAILS

```
HOST          69702dc0158e4b3ea7e406ed4c78ae13.s1.eu.hivemq.cloud
TCP PORT      8883  SSL/TLS · umqtt.simple on the ESP32
WEBSOCKET PORT 8884  SSL/TLS · MQTT.js in the browser
USERNAME      enme441
PASSWORD      enme441iot
TLS           required the cluster will refuse plaintext connections
```



- **These values already live in config.py**
hivemq.py reads MQTT_BROKER, MQTT_PORT, MQTT_USER, and MQTT_PASS from config.py.
- **Shared credentials**
The broker cannot tell one user from another since every client authenticates as enme441. Nothing about the login separates your data from anyone else's.

- **Nothing is private**
Anyone can subscribe to enme441/# and watch the data traffic. The use of user-specific topic names keeps the data streams untangled.

ENME441 Broker Topics

Hierarchical strings separated by forward slashes. The string structure is arbitrary and can be redefined as needed for different applications.



enme441/<terpmail>/sensor

ESP32 → broker

Sensor and state data, published as JSON every few seconds.

enme441/<terpmail>/commands

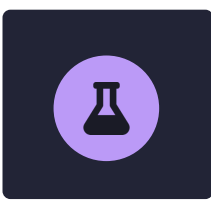
laptop → broker → ESP32

Control commands. The ESP32 subscribes here and acts in `on_message()`.

enme441/<terpmail>/status

ESP32 → broker (retained)

online / offline. Set as the Last Will so the broker publishes it if you drop.



hivemq.py — Connecting to the Broker

`start_comms()` builds the client, connects over TLS, and subscribes

```
from umqtt.simple import MQTTClient

def start_comms():
    ssl_params = {'server_hostname': config.MQTT_BROKER}
    client = MQTTClient(
        client_id = config.TERPMAIL_USERNAME,
        server     = config.MQTT_BROKER,
        port       = config.MQTT_PORT,
        user       = config.MQTT_USER,
        password   = config.MQTT_PASS,
        ssl        = True,
        ssl_params = ssl_params
    )

    client.set_callback(on_message)  # set up callback for check_msg()
    client.set_last_will(config.TOPIC_STATUS,
                        b'offline', retain=True)

    client.connect()
    client.subscribe(config.TOPIC_COMMANDS)
    client.publish(config.TOPIC_STATUS, b'online', retain=True)
    return client
```

- **Uses Transport Layer Security (TLS)**

TLS is a cryptographic security protocol that encrypts data sent between two systems over a network to keep it private.

- **set_callback()**

Define the callback function to trigger when checking for broker messages via `client.check_msg()` – see next slide.

- **set_last_will(...)**

Registered with the broker before connect. If the board drops off, the broker publishes 'offline' on your behalf.

- **Send raw bytes only**

MQTT is a binary protocol, so published payload is just a sequence of bytes (not strings). Use `b'my_string'` to convert strings to bytes.

- **subscribe before you publish**

Do it inside `start_comms()` so a reconnect always re-establishes the subscription. You can publish to a topic as many times as you like once subscribed.



hivemq.py — Receiving and Publishing

`on_message()` handles commands; `run()` keeps the connection alive

```
led = Pin(8, Pin.OUT)           # built-in LED

def on_message(topic, msg):
    data = json.loads(msg)
    cmd = data.get('cmd', '')
    if cmd == 'LED_ON':
        led.value(0)             # inverted: pull-up on LED
    elif cmd == 'LED_OFF':
        led.value(1)

def run():
    client = start_comms()
    while True:
        client.check_msg()        # non-blocking

        payload = json.dumps({'sensor_1': v1,
                              'sensor_2': v2,
                              'uptime': time.ticks_ms()})
        client.publish(config.TOPIC_SENSOR, payload)

        time.sleep(5)            # << keep-alive interval
```

- **The callback fires from `check_msg()`**

It is not a thread or an interrupt — nothing is delivered until you call `check_msg()`.

- **Extend `on_message()`, not the loop**

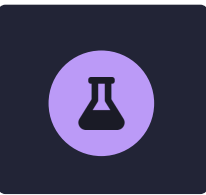
This is where every new piece of hardware in your project gets its command.

- **Payload is always in JSON format**

Use `json.dumps()` to package output to the broker and `json.loads()` to process incoming messages from the broker.

- **`sleep(5)`**

Never block more than ~30 s without calling `check_msg()` or `ping()`, or the broker treats the client as dead and disconnects it.



Testing from Your Laptop

MQTT Explorer (mqtt-explorer.com) is the easiest way to interact with the broker from your laptop.

Host `69702dc0158e4b3ea7e406ed4c78ae13.s1.eu.hivemq.cloud`

Port `8883`

Username / Password `enme441 / enme441iot` (shared course credentials)

Encryption (TLS) `enabled — required`

Publish a command from the Publish panel:

```
topic:  enme441/<terpmail_username>/commands
payload: {"cmd": "LED_ON"}
```

- **Expand the topic tree**

You should see `enme441 > your username > sensor` updating every 5 seconds.

- **Watch status too**

It should read 'online'. Unplug the board and it flips to 'offline' — that's the Last Will firing.

- **Try sending LED on/off commands**

Payload must be JSON format, using double-quotes for strings

MQTT: Things to Watch For



Connection drops after 60–90 seconds

Your loop blocks too long between `check_msg()` calls. Shorten the sleep, or pass a `keepalive` argument (the sample sets none).



MQTTException / connection refused

Check port 8883 (not 1883), verify the broker username and password, and confirm `ssl=True` is set.



ECONNABORTED on first connect

The broker can go dormant after a quiet period. Simply retry the connection.



ESP32 publishes, nothing arrives

Topics are case-sensitive and must match character for character on both ends. Confirm in MQTT Explorer.



Command sent, board ignores it

`check_msg()` is not being called, or `on_message()` is raising while parsing. Watch the Thonny REPL — every `print()` lands there.



Garbled messages or TypeError

Make sure all strings are converted to byte objects before sending to the broker. Note that `json.dumps()` and `json.loads()` will automatically convert between strings and bytes.

02

User interfacing

Interacting with the ESP32 (through through the broker) from your laptop



Interface Options



OPTION A

HTML + JavaScript

- MQTT.js over WebSocket
- Runs in any browser
- No backend at all, no file or data access (sandboxed browser)
- **Excellent UI tools and easy to share, but more complex and limited functionality.**



OPTION B

Python desktop app

- paho-mqtt + tkinter
- Python as a shared language with ESP32
- Access to files, numpy, matplotlib, etc.
- **Easy to code, excellent back-end functionality, but limited UI tools and harder to share**



OPTION C

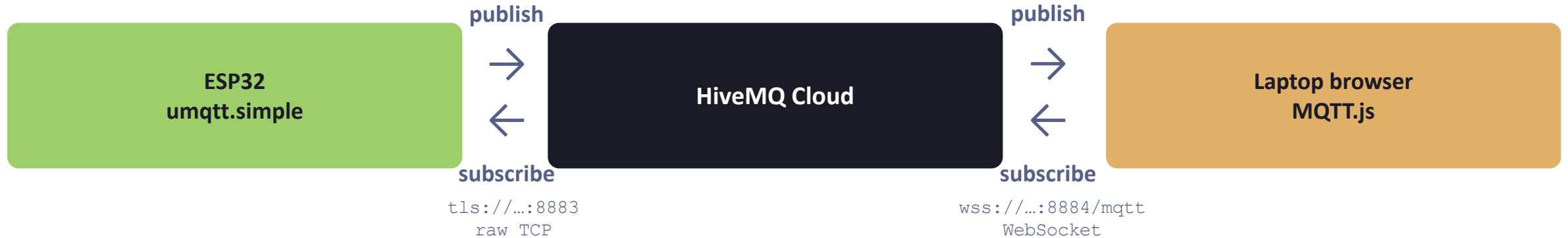
Python + HTML/js frontend

- Flask / FastAPI backend
- Credentials stay server-side
- Most moving parts
- **Best of both worlds but very high complexity**

We will use Option A as a middle ground on complexity, functionality, and UI design

Browser Uses WebSocket, not TCP

Browsers cannot open raw TCP sockets, so HiveMQ exposes a WebSocket endpoint alongside TCP.



1

The broker does not care how you connected

Messages published over TCP are delivered over WS and vice versa. Pub/sub routing is identical.

2

Both endpoints use SSL/TLS and the same credentials

Only the transport and the port number differ.

3

The /mqtt path is required

MQTT-over-WebSocket specifies it. Omit it and the connection silently fails.

4

No backend server needed

The broker IS the server. Your page is a static HTML file — open it from your local disk or host it on a web server.

MQTT Using Javascript

MQTT.js is a JavaScript/Node.js client library for the MQTT protocol. It is the JS equivalent of the `umqtt.simple` library in MicroPython that can be loaded from an online Content Delivery Network (CDN) — no Node Package Manager (npm), bundler, or build step needed

MQTT.js

in the browser · `wss://...:8884/mqtt`

```
mqtt.connect(url, {username, password})
doConnect() — url = `wss://${broker}:${port}/mqtt`

client.subscribe(['.../sensor`, `.../status`])
inside client.on('connect', ...), once the socket is up

client.on('message', (topic, msg) => {...})
event-driven — the browser fires it for you

data = JSON.parse(payload) → updateSensor()
one card per JSON key; unknown keys create a new card

client.publish(topic, JSON.stringify({cmd}))
sendCmd('LED_ON') and sendCustom() button handlers

updateSensor('status', payload)
status card; client.end(true) in doDisconnect()
```

connect

TLS on both sides

subscribe

each side listens for the other

callback

trigger action on message receipt

.../sensor

← board publishes

.../commands

browser publishes →

.../status

← board publishes, retained

umqtt.simple

on the ESP32 · `tls://...:8883`

```
MQTTClient(..., ssl=True); client.connect()
start_comms() — host, port, user, pass from config.py

client.subscribe(config.TOPIC_COMMANDS)
in start_comms(), immediately after connect()

client.set_callback(on_message)
polled — run() must keep calling client.check_msg()

client.publish(TOPIC_SENSOR, json.dumps({...}))
in run(), once every 5 s alongside check_msg()

data = json.loads(msg); data.get('cmd')
on_message() → led.value(0) / led.value(1)

client.set_last_will(TOPIC_STATUS, b'offline')
publishes b'online' on connect; broker sends offline if you drop
```

One asymmetry: *MQTT.js* is event-driven, so `on('message')` fires on its own — no `check_msg()` needed.

MQTT.js: Connecting from the Browser

```
<script src="https://unpkg.com/mqtt/dist/mqtt.min.js"></script>

const url = `wss://${brokerUrl}:${port}/mqtt`;

const client = mqtt.connect(url, {
  username: 'enme441',
  password: 'enme441iot',
  clientId: `web_${terpmail}_${Math.random()
    .toString(16).slice(2, 8)}`,
  protocolVersion: 4,
  clean: true,
  reconnectPeriod: 5000,
});
```

- **Random clientId suffix**

Two tabs would otherwise connect with the same ID and the broker would kick the first one off.

- **reconnectPeriod: 5000**

MQTT.js retries every 5 s on its own — the connect event fires again, so re-subscribe there.

- **Credentials are in the page source**

Fine for ENME441, but in production you would proxy through a backend to hide them.

MQTT.js: Subscribing and Receiving

MQTT.js is event-driven using registered callbacks.

```
client.on('connect', () => {
  client.subscribe([
    `enme441/${terpmail}/sensor`,
    `enme441/${terpmail}/status`
  ]);
});

client.on('message', (topic, message) => {
  const payload = message.toString();

  if (topic.endsWith('/sensor')) {
    const data = JSON.parse(payload);
    updateSensor('sensor_1', data.sensor_1);
    updateSensor('uptime', data.uptime);
  }

  if (topic.endsWith('/status')) {
    document.getElementById('val-status')
      .textContent = payload;
  }
});
```

- **Subscribe inside on('connect')**

Guarantees the connection exists first — and re-subscribes automatically after any reconnect.

- **The payload is a Buffer**

Call toString() before you do anything with it, then JSON.parse() if it's JSON.

- **Route on the topic**

endsWith('/sensor') tells you what kind of message arrived without parsing the whole topic string.

- **Same JSON both ways**

Whatever keys hivemq.py put in json.dumps() are exactly the keys you read here.

MQTT.js: Publishing Commands Back

One line of JavaScript closes the loop to your `on_message()` callback on the board.

```
function sendCmd(cmd) {  
  const topic = `enme441/${terpmail}/commands`;   
  const payload = JSON.stringify({ cmd: cmd });  
  client.publish(topic, payload);  
}
```

```
<button onclick="sendCmd('LED_ON')">LED On</button>  
<button onclick="sendCmd('LED_OFF')">LED Off</button>
```

...and on the ESP32, already waiting for it:

```
def on_message(topic, msg):  
    data = json.loads(msg)  
    if data.get('cmd') == 'LED_ON':  
        led.value(0)
```

- **publish(topic, payload)**

That is the entire API. The broker delivers it to every subscriber of that topic.

- **JSON.stringify ↔ json.loads**

Both ends agree on JSON, so adding a field means adding a key — no protocol to redesign.

- **Add a parameter, not a topic**

{ cmd: 'SERVO', angle: 90 } carries data alongside the command. Handle it in the same callback.

- **Custom JSON needs double quotes**

The dashboard's free-text box passes your string straight through, and `json.loads()` is strict about it.



Interface Template: mqtt_dashboard.html

One HTML file with embedded Javascript, no dependencies beyond MQTT.js. Opens in any browser with Javascript enabled.

1

Connection panel

Broker, port 8884, username and password are pre-filled for the course. Type your terpmail username and click Connect — the indicator turns green.

2

Live sensor cards

Updates as messages arrive, with no page refresh. A new card will be created automatically for any JSON key the template doesn't recognize.

3

Command controls

LED On / LED Off buttons, plus a free-text JSON field for sending anything else to your board.

4

Scrolling message log

Every received message with a timestamp. Pair it with the browser console (F12 / Cmd-Opt-I) for debugging when something isn't working.

Extending the Template

The template is a foundation for you to build on. Here are a couple examples of how you might extend the code.

```
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

// in the message handler:
if (topic.endsWith('/sensor')) {
  const data = JSON.parse(payload);
  readings.push({ time: new Date(), value: data.sensor_1 });
  if (readings.length > 60) readings.shift();
  updateChart(readings);
}
```

● Add a chart

Keep a rolling array of the last N readings and redraw on each message. Chart.js loads from a CDN like MQTT.js does.

```
<input type="range" min="0" max="180" id="servo">
<button onclick="sendServo()">Set Angle</button>

function sendServo() {
  const angle = document.getElementById('servo').value;
  client.publish(`enme441/${terpmail}/commands`,
    JSON.stringify({ cmd: 'SERVO', angle: Number(angle) }));
}
```

● Add a control

New button or slider → sendCmd() with a new command string → a new branch in on_message() on the ESP32.

- **Extend further using a framework**, e.g. React or Vue — npm install mqtt and import it. Only the rendering layer changes.

What to Remember

Nothing on either side knows the other exists — the broker and a topic string are the entire interface.



Outbound only, both sides

umqtt-simple blocks every inbound connection. The ESP32 and the browser each dial out to HiveMQ and neither one ever accepts a connection.



Topics are the only contract

enme441/terpmail/sensor, /commands, /status. The board and the browser never see each other — they only agree on these strings.



One protocol, two libraries

umqtt-simple over TLS on 8883, MQTT.js over WebSocket on 8884. Same credentials, same topics; the broker bridges the two transports.



Event-driven vs. polled

on('message') fires by itself in the browser. On the ESP32, run() must keep calling check_msg() or the broker treats the client as dead.



JSON in, JSON out

json.dumps() on the board, JSON.parse() in the page. Adding a sensor is adding a key — the dashboard builds a new card for it on its own.



mqtt_dashboard.html is a starting point

One HTML file, no backend, no build step. Extend it with charts, sliders, or anything else your project needs.

Lab 10 – Brokered Communication

